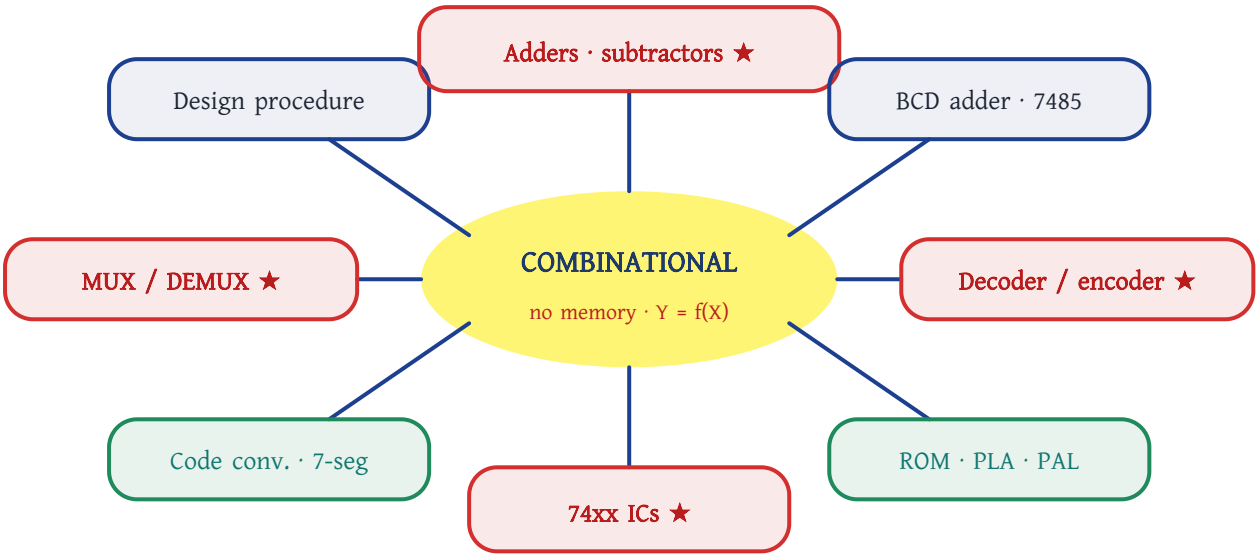
# Combinational Circuits

Adders · CLA · BCD · comparator · decoder · MUX · PLA: exam-ready in 11 sheets




↑ the whole topic on one page: red = high-yield, green = applied / programmable

## Syllabus coverage

"Digital logic: … **combinational logic circuits**, minimization techniques; **Arithmetic and data handling logic circuits, decoders and encoders, multiplexers and demultiplexers**; **Combinational circuit design**; … **PLA design**". This notebook covers all of these. Flip-flops and counters are in the Sequential Circuits notes; K-maps are in the Boolean Algebra notes.

**High-yield list**

① HA / FA equations
② FA = 2 HA + 1 OR
③ CLA: G and P terms
④ n selects → $2^n$ inputs
⑤ decoder = minterms
⑥ 7483 · 74138 · 74151

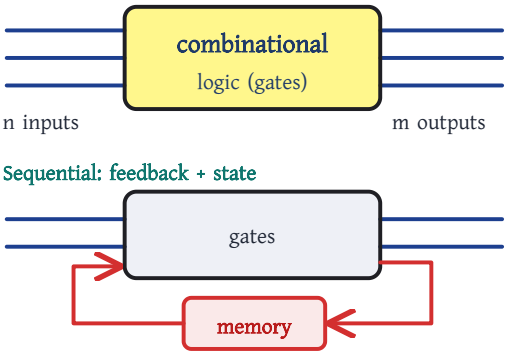
**Legend:** **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

## Contents


✓ Basics, design, half adder ..... p. 2
✓ Full adder & subtractors ..... p. 3
✓ Ripple-carry & look-ahead ..... p. 4
✓ Adder-subtractor, BCD, comparator … p. 5
✓ Decoders & encoders ..... p. 6
✓ MUX & DEMUX ..... p. 7
✓ Code converters, parity, 7-seg … p. 8
✓ ROM · PLA · PAL · hazards ..... p. 9
✓ Night-before sheet ..... p. 10
✓ Practice MCQs + written ..... p. 11

## 1 Combinational vs Sequential

**Definition:** A **combinational circuit** is a network of gates whose outputs depend **only on the present inputs**: Y = f(X). No memory, no clock, no feedback. A **sequential** circuit also uses **stored state** (flip-flops) → Sequential Circuits notes.




top: Y = f(X) · bottom: Y = f(X, state)

| Feature | Combinational | Sequential |
|---|---|---|
| Memory | none | flip-flops |
| Clock | not needed | usually |
| Feedback | none | yes |
| Model | truth table | state table |
| Examples | adder, MUX | counter, register |

## 2 Design Procedure

1. Fix the **inputs and outputs**; name them
2. Write the **truth table** ($2^n$ rows; unused codes = X)
3. Simplify each output (K-map, Boolean Algebra sheets 7–8)
4. Draw the circuit (AND-OR → NAND-NAND)
5. Verify a few rows

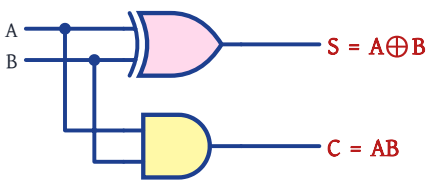
**Worked example ✍️:** 3-bit **prime detector**:
F = 1 when ABC (A = MSB) is 2, 3, 5 or 7.
F = Σm(2, 3, 5, 7): pairs m2, m3 = **A′B**; m5, m7 = **AC**
F = **A′B + AC** · check 001: 0 ✓

**Families:** **Arithmetic:** adders, subtractors, comparators, ALU · **Data routing:** MUX, DEMUX, decoders, encoders · **Code conversion:** BCD↔XS-3, binary↔Gray, 7-segment · **Error detection:** parity · **Programmable:** ROM, PLA, PAL

## 3 Half Adder (HA)

| A | B | S | C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

Adds **two** bits; no carry-in.
Sum **S = A ⊕ B**
Carry **C = A · B**
1 XOR + 1 AND (or 5 NAND)



Memory trick: **Sum = XOR** ("exactly one 1"), **Carry = AND** ("both 1"): $1 + 1 = 10_2$.

## 4 Full Adder (FA)

| m | A | B | $C_{in}$ | S | $C_{out}$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | 0 |
| 2 | 0 | 1 | 0 | 1 | 0 |
| 3 | 0 | 1 | 1 | 0 | 1 |
| 4 | 1 | 0 | 0 | 1 | 0 |
| 5 | 1 | 0 | 1 | 0 | 1 |
| 6 | 1 | 1 | 0 | 0 | 1 |
| 7 | 1 | 1 | 1 | 1 | 1 |

Adds **three** bits: A, B and the carry-in from the previous stage.

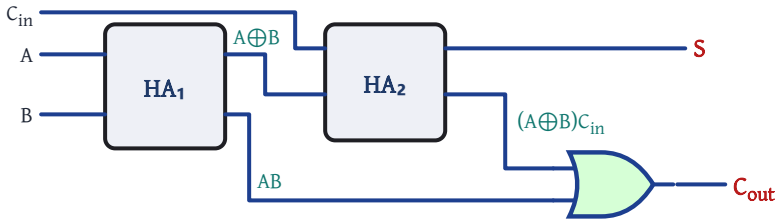
S = Σm(1, 2, 4, 7) → 1 when the number of 1s is **odd**:

**$S = A \oplus B \oplus C_{in}$**

$C_{out}$ = Σm(3, 5, 6, 7) → K-map gives three pairs:

**$C_{out} = AB + BC_{in} + AC_{in}$**

**$= AB + C_{in}(A \oplus B)$** (the form used by two half adders) = majority of A, B, $C_{in}$




FA = two half adders + one OR gate ($C_{out} = C_1 + C_2$)

| Circuit | With basic gates | NAND only | NOR only |
|---|---|---|---|
| HA / HS | 1 XOR + 1 AND (+1 NOT for HS) | 5 | 5 |
| FA / FS | 2 XOR + 2 AND + 1 OR (2 halves + OR) | 9 | 9 |

## 5 Half & Full Subtractor

| A | B | D | $B_o$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |

| A | B | $B_{in}$ | D | $B_{out}$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

**Half subtractor** (A − B):

**$D = A \oplus B$**

**$B_o = A'B$**

(borrow only for 0 − 1)

**Full subtractor** (A − B − $B_{in}$): borrow when A < B + $B_{in}$.

FS = two half subtractors + one OR ($B_{out} = B_1 + B_2$).

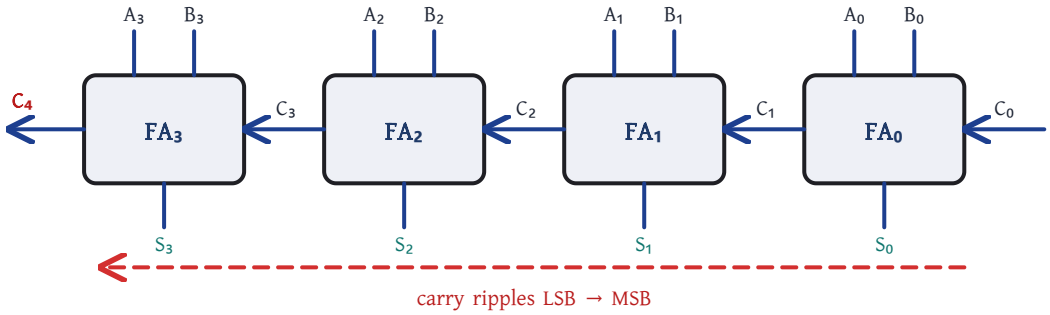
FS: $D = A \oplus B \oplus B_{in} = \Sigma m(1, 2, 4, 7) \cdot B_{out} = \Sigma m(1, 2, 3, 7) = \mathbf{A'B + A'B_{in} + BB_{in}} = A'B + B_{in}(A \oplus B)'$

**Memory trick:** Difference = Sum (same XOR). Borrow = carry with **A complemented**: $AB \rightarrow A'B$, $AC_{in} \rightarrow A'B_{in}$.

**Exam trap:** FA = 2 HA + **OR** (not AND). $C_{out} \neq A \oplus B \oplus C_{in}$: that is the sum. HA cannot take a carry-in.

## 6 Ripple-Carry (Parallel) Adder

**Definition:** An **n-bit parallel adder** chains **n full adders**: each $C_{out}$ feeds the next $C_{in}$ ($C_0 = 0$). All bits are applied at once, but the carry **ripples** from LSB to MSB.




**Delay:** Carry path per stage = 2 gate levels (AND → OR); worst case it passes **all n stages**: $t \approx \mathbf{n \times t_{carry}} \approx 2n$ gate delays. 16 bits × 10 ns = **160 ns**.

**IC 7483** (newer pinout **74283**): 4-bit binary adder with fast carry; cascade two for 8 bits ($C_4 \rightarrow$ next $C_0$).

Cheap (n FAs) but **slow**: delay grows linearly with n.

## 7 Carry Look-Ahead Adder (CLA)

**Idea:** Compute every carry **directly from the inputs**, no ripple. Per bit: $\mathbf{G_i = A_iB_i}$ (generate: makes a carry itself), $\mathbf{P_i = A_i \oplus B_i}$ (propagate: passes $C_i$ on).

**Equations:** $C_{i+1} = G_i + P_iC_i \cdot S_i = P_i \oplus C_i$

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

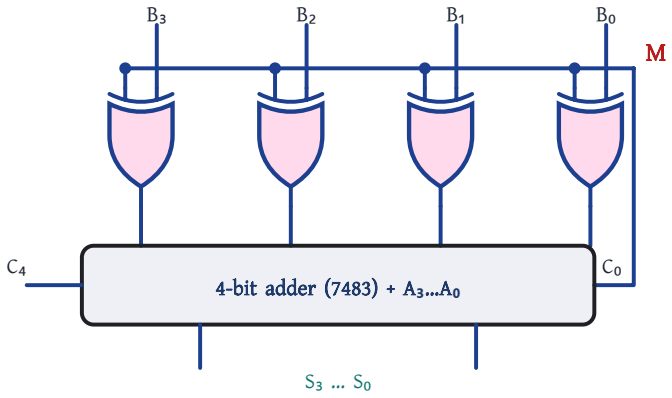
| | Ripple | Look-ahead |
|---|---|---|
| Carry delay | ≈ 2n | 3 (fixed) |
| Gates | few | many |
| Fan-in | 2–3 | up to n + 1 |
| IC | 7483 | 74182 |

**Worked example ✍️:** A = 1011, B = 0101, $C_0 = 0 \rightarrow G = 0001$, $P = 1110 \cdot C_1 = G_0 = 1$, $C_2 = P_1C_1 = 1$, $C_3 = 1$, $C_4 = 1$ (one carry propagated three places, yet all four appear together) · $S = P \oplus C_3C_2C_1C_0 = 1110 \oplus 1110 = 0000 \rightarrow$ **1 0000** (11 + 5 = 16 ✓)

**Memory trick:** G = "both 1, I **G**ive a carry" · P = "one 1, I **P**ass it on". 3 levels: P/G → AND → OR.

**Exam trap:** CLA delay is fixed only in theory: fan-in grows with n, so real adders use 4-bit CLA blocks + the 74182.

## 8 4-bit Adder-Subtractor (XOR trick)




Each $B_i$ goes through an XOR with mode line **M**; $C_0$ = M:

**M = 0** → B ⊕ 0 = B, $C_0$ = 0 → **A + B**

**M = 1** → B ⊕ 1 = B′, $C_0$ = 1 → A + B′ + 1 = **A − B** (2's complement)

Signed overflow: **V = $C_4$ ⊕ $C_3$**

Unsigned: $C_4$ = 1 → A ≥ B; $C_4$ = 0 → negative result.

**Worked example ✍️:** 7 − 5: 0111 + 1010 + 1 = **1 0010** → 2 (drop $C_4$) · 5 − 7: 0101 + 1000 + 1 = **0 1110** = −2 · 6 + 5 = 1011: $C_3$ = 1, $C_4$ = 0 → V = 1, **overflow**

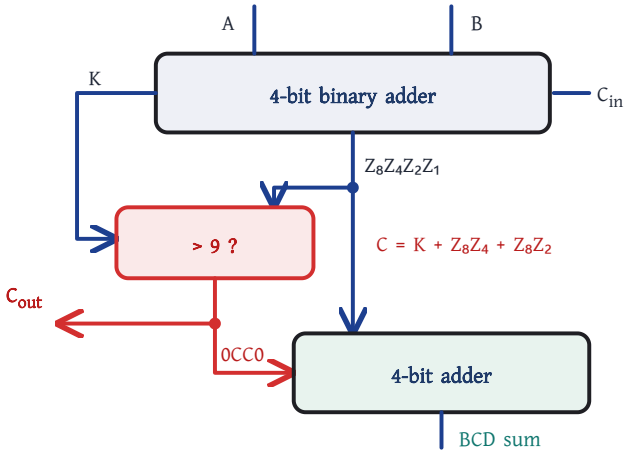
## 9 BCD Adder

Adds two BCD digits + carry. A 4-bit binary adder gives K $Z_8Z_4Z_2Z_1$ (max 9 + 9 + 1 = 19).

If the sum **> 9** (or K = 1), **add 0110 (6)** to skip the 6 unused codes:

**C = K + $Z_8Z_4$ + $Z_8Z_2$**

Two 4-bit adders + correction logic (2 AND + 1 OR); C is the decimal carry out.




**Worked example ✍️:** 8 + 7 = 1111 → + 0110 = 1 0101 → **0001 0101** = 15 ✓ · 9 + 9 = 1 0010 (K = 1) → 0010 + 0110 = 1000 → **0001 1000** = 18 ✓ · 4 + 5 = 1001: no correction

## 10 Magnitude Comparator

| A | B | A > B | A = B | A < B |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 |

**1-bit:** $(A > B) = AB' \cdot (A < B) = A'B \cdot (A = B) = \mathbf{A \odot B}$ (XNOR)

**4-bit:** $x_i = A_i \odot B_i \cdot \mathbf{(A = B) = x_3x_2x_1x_0}$

$(A > B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$

$(A < B)$: same pattern with $A_i'B_i$. Compare from the **MSB** down.

**IC 7485**: 4-bit magnitude comparator with cascade inputs (>, =, <); for a single chip tie "=" HIGH. Example: A = 1011, B = 1001 → $x_3 = x_2 = 1$, $A_1B_1' = 1$ → **A > B** (11 > 9 ✓).

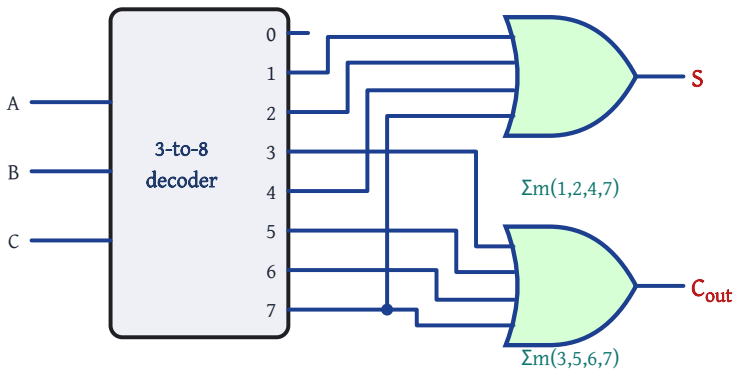
## 11 Decoder (n → $2^n$)

**Definition:** A decoder converts an n-bit code into **$2^n$ output lines**, exactly one active (e.g. memory **chip select**). Each output is one **minterm**, so a decoder is a **minterm generator**.

| E | A | B | $D_0$ | $D_1$ | $D_2$ | $D_3$ |
|---|---|---|---|---|---|---|
| 0 | X | X | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 1 |

2-to-4 with enable: $\mathbf{D_i = E \cdot m_i}$ ($D_0 = EA'B'$ … $D_3 = EAB$). 3-to-8 = 3 NOT + 8 AND.

**74138**: 3-to-8, **active-LOW** outputs, enables G1 (high), G2A, G2B (low) · **74139**: dual 2-to-4 · **74154**: 4-to-16 · **7442**: BCD-to-decimal.




full adder = one 3-to-8 decoder + two 4-input OR gates

**Any function:** n-to-$2^n$ decoder + one OR gate per output (OR the minterms). With active-LOW outputs (74138) use a **NAND** gate instead.

**Exam trap:** 4-to-16 from 3-to-8: **2** decoders ($A_3$ drives the enables). 4-to-16 from 2-to-4: 4 + 1 = **5**.

## 12 Encoder & Priority Encoder ($2^n$ → n)

Reverse of a decoder. **8-to-3** (octal-to-binary), one input active at a time; just 3 OR gates:

**Priority encoder**: the highest active input wins; **V** (valid) = 1 if any input is active. $D_3$ highest:

$A_2 = D_4 + D_5 + D_6 + D_7$

$A_1 = D_2 + D_3 + D_6 + D_7$

$A_0 = D_1 + D_3 + D_5 + D_7$

**Two faults:** $D_3$ and $D_6$ together → 111 (wrong); "$D_0$ active" and "nothing active" both give 000.

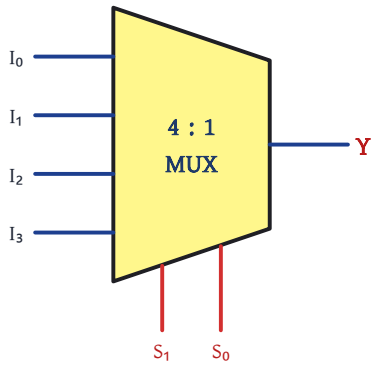
| $D_0$ | $D_1$ | $D_2$ | $D_3$ | x | y | V |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | X | X | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| X | 1 | 0 | 0 | 0 | 1 | 1 |
| X | X | 1 | 0 | 1 | 0 | 1 |
| X | X | X | 1 | 1 | 1 | 1 |

x = $\mathbf{D_2 + D_3}$ · y = $\mathbf{D_3 + D_1D_2'}$ · V = $D_0 + D_1 + D_2 + D_3$ · **74148**: 8-to-3 priority encoder (active-LOW, input 7 highest) · **74147**: 10-to-4 (decimal-to-BCD). Use: interrupt priority in a CPU, keyboard encoding.

> **Memory trick:** **DE**coder **DE**-compresses: n → $2^n$. **EN**coder compresses: $2^n$ → n. The side with more lines is "one-hot".

## 13 Multiplexer (MUX): Data Selector

> **Definition:** A MUX routes **one of $2^n$ data inputs** to a single output; **n select lines** choose which. Many-to-one, "data selector", **parallel-to-serial** converter.




**2:1:** $Y = S'I_0 + SI_1$

**4:1:** $Y = \mathbf{S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3}$

4:1 gates: 2 NOT + 4 AND + 1 OR.

| Inputs | 2 | 4 | 8 | 16 | 32 |
|---|---|---|---|---|---|
| Select lines | 1 | 2 | 3 | 4 | 5 |

**ICs: 74151** 8:1 (outputs Y and Y′) · **74153** dual 4:1 · **74157** quad 2:1 · **74150** 16:1

### Implementing a function with a MUX

An n-variable function needs only a **$2^{n-1}$ : 1 MUX**: n − 1 variables drive the selects; each data input is **0, 1, x or x′** (x = last variable).

(A $2^n$:1 MUX works directly: $I_i$ = the truth-table output, 0 or 1.)

**Worked example ✍️:** F(A, B, C) = Σm(1, 2, 6, 7), 4:1 MUX, $S_1S_0$ = AB

| AB | C = 0 | C = 1 | I |
|---|---|---|---|
| 00 | 0 | 1 | C |
| 01 | 1 | 0 | C′ |
| 10 | 0 | 0 | 0 |
| 11 | 1 | 1 | 1 |

$I_0 \ldots I_3$ = **C, C′, 0, 1**

**MUX trees:** 16:1 from 4:1 → 4 + 1 = **5**
16:1 from 2:1 → 8 + 4 + 2 + 1 = **15** · 8:1 from 2:1 → **7**
$2^k$:1 from 2:1 MUXes → $2^k - 1$

**Memory trick:** **MUX = many in, one out**: a rotary switch set by the select number. Selects = $\log_2$(inputs).

## 14 Demultiplexer (DEMUX): Data Distributor

One input D → **one of $2^n$ outputs**, chosen by n selects. One-to-many, **serial-to-parallel**.

1:4 → $Y_0 = S_1'S_0'D$, $Y_1 = S_1'S_0D$, $Y_2 = S_1S_0'D$, $Y_3 = S_1S_0D$

A **decoder with enable = DEMUX**: feed the data into E, the selects into the code inputs. So the **74138** also works as 1:8 DEMUX, and the **74155** is a dual 2-to-4 decoder/DEMUX.

**Exam trap:** A MUX has **1 output**, a DEMUX **1 input**. 16:1 from 4:1 needs 5, not 4. A MUX is a **universal logic module** (any function).

**IT link:** A MUX picks the ALU operand in a CPU; TDM links = MUX + DEMUX.

## 15 Code Converters

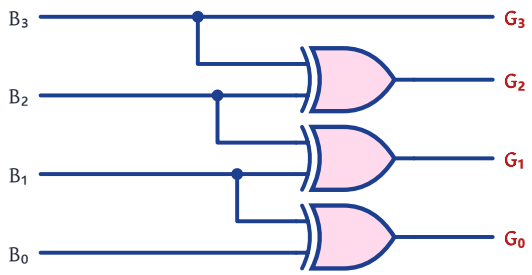
**BCD → Excess-3** (XS-3 = BCD + 0011; codes 10–15 are don't-cares). Inputs ABCD (A = MSB), outputs wxyz:

**$z = D'$** · $y = CD + C'D' = (C \oplus D)'$
$x = B'C + B'D + BC'D'$ · $w = A + BC + BD$

Check 5 = 0101 → w = BD = 1, x = 0, y = 0, z = 0 → **1000** = 8 ✓

**Binary → Gray:** $G_3 = B_3$, $G_i = B_{i+1} \oplus B_i$ (n − 1 XORs)
**Gray → binary:** $B_3 = G_3$, $B_i = B_{i+1} \oplus G_i$ (uses the new B)

1011 → **1110** (Gray) and back: 1, 1⊕1 = 0, 0⊕1 = 1, 1⊕0 = 1 → 1011 ✓



binary → Gray: copy the MSB, XOR neighbours

## 16 Parity Generator & Checker

**Even-parity generator** (3-bit message ABC): **$P = A \oplus B \oplus C$**, so the total number of 1s (with P) is even. Odd parity: $P = (A \oplus B \oplus C)'$.

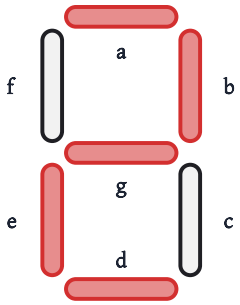
n bits → n − 1 two-input XOR gates (a "parity tree"). ICs: **74180**, **74280** (9-bit generator/checker).

Example: 101 → P = 0, send 1010; receive 1110 → PEC = 1 → error ✓

**Checker** (receiver): PEC = A ⊕ B ⊕ C ⊕ P; **0 = OK, 1 = error.**

> **Exam trap:** Parity detects any **odd** number of bit errors, but **not** 2 (or any even number) and it cannot correct (Hamming code can).

## 17 BCD-to-7-Segment Decoder




digit 2: a b d e g

| Digit | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| Segments lit | 6 | 2 | 5 | 5 | 4 | 5 | 6 | 3 | 7 | 6 |

4 BCD inputs → 7 outputs (a–g), one function per segment; codes 10–15 = don't-cares. Segment a is off only for 1 and 4: **a = A + C + BD + B′D′**

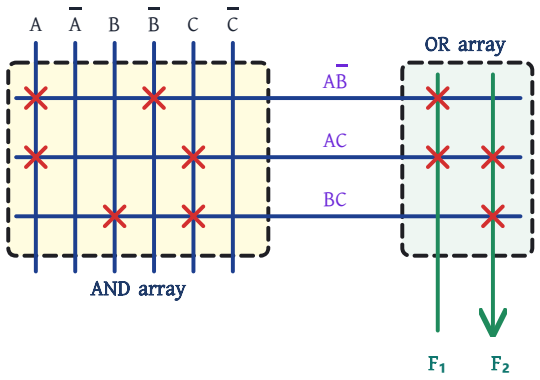
**7447**: active-LOW (open-collector) outputs → **common-anode** display · **7448**: active-HIGH → **common-cathode** · CMOS: 4511.

> **Memory trick:** **"47 is LOW, Anode"**: 7447 pulls a segment LOW to light it, so the common pin goes to +V (anode). 7448 is the opposite.

## 18 ROM, PLA, PAL

> **Idea:** Any SOP = an **AND array** (product terms) feeding an **OR array**. PLDs differ only in **which array is programmable**.

| Device | AND array | OR array | Key point |
|---|---|---|---|
| ROM / PROM | fixed (decoder) | programmable | all $2^n$ minterms; stores the truth table as is |
| PLA | programmable | programmable | most flexible; product terms **shared** |
| PAL | programmable | fixed | cheaper, faster; no sharing (GAL: erasable) |




PLA: × = programmed link; AC is shared

$F_1$ = AB′ + AC, $F_2$ = AC + BC: a PLA needs **3** product terms (AC shared); a PAL needs 4.

**ROM size $2^n$ × m**: 32 × 8 → 5 address lines, 8 outputs, 256 bits. Full adder = 8 × 2 ROM.

**PLA size** n × k × m (inputs × products × outputs): minimise **distinct product terms**, not literals.

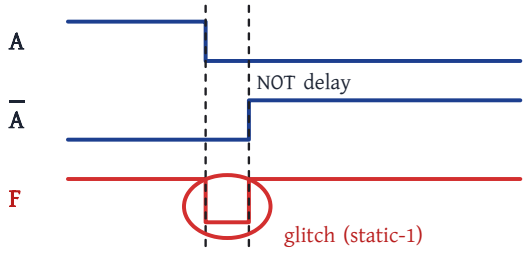
Modern: CPLD, **FPGA** (look-up tables).

**Memory trick:** **ROM**: AND fixed · **PAL**: OR fixed (P-A-L: "**A**ND is programmable, the **L**ast stage is locked") · **PLA**: nothing fixed.

## 19 Hazards (glitches)

A **hazard** is a momentary wrong output caused by unequal path delays.

- **Static-1:** should stay 1, dips to 0 (SOP)
- **Static-0:** should stay 0, jumps to 1 (POS)
- **Dynamic:** changes 3+ times instead of once




B = C = 1, A falls: F should stay 1

F = AB + A′C, B = C = 1: when A falls, AB turns off before A′C turns on. **Fix:** add the consensus term **BC** (Boolean Algebra notes, sheet 3), i.e. cover every pair of adjacent 1s by one K-map group.

**Exam trap:** The redundant term a minimal K-map drops is exactly what removes the hazard. Clocked systems sample after outputs settle, so hazards matter most in asynchronous circuits.

# Night-Before-the-Exam Sheet

**Fact sheet:**

Combinational: Y = f(present X), no memory

n inputs → $2^n$ truth-table rows

HA: S = A⊕B · C = AB

FA: S = A⊕B⊕$C_{in}$ · $C_{out}$ = AB + $C_{in}$(A⊕B)

HS: D = A⊕B · $B_o$ = A′B

FS: $B_{out}$ = A′B + A′$B_{in}$ + B$B_{in}$

FA = 2 HA + OR · 9 NAND · HA = 5 NAND

Ripple delay ∝ n (≈ 2n gate delays)

CLA: G = AB, P = A⊕B, $C_{i+1}$ = $G_i$ + $P_iC_i$

Add-sub: B ⊕ M, $C_0$ = M · V = $C_4$ ⊕ $C_3$

BCD: add 0110 if K + $Z_8Z_4$ + $Z_8Z_2$

Comparator: A = B ⇔ all $x_i$ = $A_i$ ⊙ $B_i$ = 1

Decoder n → $2^n$ · encoder $2^n$ → n

MUX: $2^n$ inputs, n selects · DEMUX reverse

n-var function → $2^{n-1}$ : 1 MUX

Gray: $G_i$ = $B_{i+1}$ ⊕ $B_i$ · XS-3 = BCD + 0011

Even parity P = XOR of the data bits

ROM: fixed AND · PAL: fixed OR

## 74xx ICs to know

| IC | Function | IC | Function |
|---|---|---|---|
| **7483** | 4-bit adder (also 74283) | **74151** | 8:1 MUX |
| 74182 | look-ahead carry generator | 74153 | dual 4:1 MUX |
| 74181 | 4-bit ALU | 74157 · 74150 | quad 2:1 · 16:1 MUX |
| 7485 | 4-bit comparator | 74155 | dual 2-to-4 decoder/DEMUX |
| **74138** | 3-to-8 decoder, active-LOW | 74148 | 8-to-3 priority encoder |
| 74139 · 74154 | dual 2-to-4 · 4-to-16 | 74147 | 10-to-4 priority encoder |
| 7442 | BCD-to-decimal | 7447 · 7448 | BCD-to-7-seg (CA · CC) |
| 74180 · 74280 | parity gen./checker | 4511 (CMOS) | BCD-to-7-seg, CC |

**Top traps:** FA = 2 HA + OR (not AND) · difference = sum (A⊕B⊕C) · 16:1 from 4:1 = 5 · BCD correction is +6, XS-3 is +3 · plain encoder fails with 2 active inputs · 74138 / 7447 outputs are active-LOW · parity misses 2-bit errors

**All tricks together:** Sum = XOR, carry = AND · borrow = carry with A′ · G gives, P passes · MUX = many in, one out; selects = $\log_2$(inputs) · DEcoder DE-compresses · "over nine? add six" · 47 is LOW, anode · ROM fixed AND, PAL fixed OR, PLA nothing fixed · Gray: copy MSB, XOR neighbours



the adder family: each step builds on the last

## ✓ Practice MCQs (exam pattern, 10 marks)

**1.** The carry output of a half adder is:
(a) A ⊕ B (b) A + B
(c) A · B (d) (A · B)′

**2.** Select lines of a 32:1 MUX:
(a) 5 (b) 4
(c) 32 (d) 16

**3.** Minimum 2-input NAND gates for a full adder:
(a) 5 (b) 7
(c) 8 (d) 9

**4.** Output lines of a decoder with 4 inputs:
(a) 8 (b) 16
(c) 4 (d) 2

**5.** In a carry look-ahead adder, the generate term $G_i$ is:
(a) $A_i \oplus B_i$ (b) $A_i + B_i$
(c) $A_iB_i$ (d) $(A_iB_i)'$

**6.** A BCD adder corrects a sum above 9 by adding:
(a) 0011 (b) 0110
(c) 1010 (d) 1001

**7.** Which IC is an 8-to-1 multiplexer?
(a) 74138 (b) 7485
(c) 7447 (d) 74151

**8.** Programmable AND + fixed OR array:
(a) PAL (b) PROM
(c) PLA (d) RAM

**9.** How many 4:1 MUXes make a 16:1 MUX?

(a) 4 (b) 5

(c) 8 (d) 3

**10.** XOR adder-subtractor, for A − B:

(a) M = 0, $C_0$ = 0 (b) M = 1, $C_0$ = 0

(c) M = 1, $C_0$ = 1 (d) M = 0, $C_0$ = 1

Answer key → 1-c · 2-a · 3-d · 4-b · 5-c · 6-b · 7-d · 8-a · 9-b · 10-c

**Written question:** Implement F(A, B, C, D) = Σm(1, 3, 4, 11, 12, 13, 14, 15) with an 8:1 multiplexer (A, B, C on the select lines, A = MSB). 8 marks

1. Selects = A, B, C; each data input is 0, 1, D or D′ 1
2. For each ABC, list F at D = 0 and D = 1 (table) 3
3. (0, 1) → D · (1, 0) → D′ · (0, 0) → 0 · (1, 1) → 1, so $I_0$…$I_7$ = **D, D, D′, 0, 0, D, 1, 1** 2
4. One 74151 + one NOT for D′; check m12: $I_6$ = 1 ✓ 2

| ABC | D=0 | D=1 | I |
|---|---|---|---|
| 000 | 0 | 1 | D |
| 001 | 0 | 1 | D |
| 010 | 1 | 0 | D′ |
| 011 | 0 | 0 | 0 |
| 100 | 0 | 0 | 0 |
| 101 | 0 | 1 | D |
| 110 | 1 | 1 | 1 |
| 111 | 1 | 1 | 1 |

## Before the exam ✓

- ✓ HA, FA, HS, FS tables and equations
- ✓ FA from two HAs and from a decoder
- ✓ CLA carries $C_1$…$C_4$; MUX data inputs
- ✓ 7483, 74138, 74151, 7485, 74148, 7447

**Also practise (written):**

① BCD adder block diagram
② 4-bit comparator equations
③ BCD → XS-3 with K-maps
④ FA with a 3-to-8 decoder